

The Runic System

Technical Guide

A plain-language explanation of how the classroom rune-tracking app works, plus the technical reference a future Claude session needs to safely extend it.

Covers: **index.html** (frontend, hosted from GitHub on Render) + **Code.gs** (backend) + Google Sheet backend, including the **Students**, **Pins**, **Config**, and **Runewords** tabs.

Also documents: **runic-system.html**, the offline single-device fallback version (unchanged by anything in this revision).

Revision note: this edition adds the teacher-managed item catalog ("Manage Items": add / edit / hide / delete runewords from the app itself), the God Rune forging recipe that replaced hand-granting, and documents the current GitHub + Render hosting setup and the baked-in backend URL. It supersedes the previous edition's Sections 5.4, 7, 8, and 9, which described runewords as hardcoded and duplicated across both files — that is no longer how the app works.

Read this first if you are Claude

This document is written to be pasted into a future conversation as context before making changes to the Runic System app. Section 9 ("Notes for future Claude sessions") is aimed specifically at you: it lists where each piece of logic lives, what must stay in sync between files, and the deployment steps a change requires.

Skim Sections 2–5 for the architecture and data model before editing anything. The single most important architectural fact in this revision: **runeword game content (names, recipes, descriptions, photos) now lives only in the Google Sheet's Runewords tab, not in either code file.** The old "duplicated on purpose" rule from the previous edition of this guide (Section 5.4) applies now only to the twelve base runes' stats — not to runewords. See Section 5.4 and Section 9.2 for exactly what that changes about how you should make content edits.

1. What this app is, in plain terms

The Runic System is a classroom reward tool for a Dungeons & Dragons campaign. Students earn runes — twelve different kinds, each with its own rarity — by redeeming one-time codes the teacher hands out at quest milestones. Runes can then be combined into runewords: magic items usable in the tabletop game (a ring, a sword, a pair of boots, and so on), each requiring a specific combination of runes to craft.

The app is a website with three main things a student can do, plus a teacher-only control panel:

- **Roll** — redeem a one-time milestone code for three random runes.
- **Transform** — trade in a combination of lower-value runes to forge a specific higher-value rune, **or** destroy one of each of the twelve ordinary runes at once to forge a God Rune (new — see Section 6.1).
- **Craft** — spend a specific set of runes to permanently forge a named magic item (a "runeword").
- **Teacher tools** — generate milestone codes, manage the entire runeword catalog (add, edit, hide, or permanently delete items — new, see Section 6.2), see every student's progress, and reset a forgotten PIN.

Every student's rune count lives in one shared, always-up-to-date place, so it doesn't matter whether a student checks their runes on a classroom Chromebook, their own phone, or the teacher's laptop — everyone sees the same numbers. The same is now true of the runeword catalog itself: an item the teacher adds or edits from any device appears for every student immediately, with no code edit or redeploy required.

2. How the pieces fit together

There is no traditional “app server.” Instead, three ordinary pieces of infrastructure work together:

Piece	What it is	Where it lives
Frontend	A single self-contained HTML file (index.html) with all its CSS and JavaScript inline, plus an images/ folder of PNGs for seeded runewords. This is what a student's browser actually loads and displays.	A GitHub repository, deployed as a static site on Render (visiirunes.onrender.com). Pushing to GitHub auto-redeploys.
Backend	A small program (Google Apps Script, Code.gs) that receives requests from the frontend, applies the game rules, and reads/writes the spreadsheet.	Runs inside Google's infrastructure, attached to a Google Sheet.
Database	An ordinary Google Sheet with four tabs (Students , Pins , Config , Runewords) used as a simple row-based database.	Lives in the teacher's Google Drive.

The request flow for, say, rolling for runes, looks like this:

Student's browser	Google Apps Script	Google Sheet
-----	-----	-----
1. Click "Roll"		
2. fetch() POST ----->	2. doPost() receives it	
	3. Checks the student's PIN --> reads Students tab	
	4. Checks the milestone code -> reads Pins tab	
	5. Rolls 3 random runes	
	6. Writes new totals -----> writes Students tab	
	7. Marks the code as used ----> writes Pins tab	
8. Shows the 3 runes <-----	Returns JSON result	

Every single action (rolling, transforming, crafting, generating a code, adding or editing an item) follows this same shape: the browser sends a small JSON message describing what it wants to do, the Apps Script backend is the only thing that actually checks the rules and touches the spreadsheet, and it sends back the correct, updated data. The browser never edits the spreadsheet directly and never decides on its own whether an action is legal — it just displays whatever the backend says is true.

2.1 Why it's built this way (not, say, a normal database)

- **No server to pay for or maintain.** Google Apps Script and Google Sheets are free, and Google handles hosting, uptime, and security patching. The static frontend on Render's free tier is likewise free to host.
- **The teacher already owns and can read the data at any time**, just by opening the Sheet — no separate admin tool needed to inspect or manually fix something, including individual runeword rows now.
- **Good enough concurrency for a classroom.** A short server-side lock (Section 5.6) makes sure two students (or a student and the teacher editing an item) acting at the exact same instant can't corrupt each other's data.

3. The three kinds of credentials

It's easy to confuse these three, since they're all short numeric codes (or, for the teacher, a short passphrase). They are completely separate and serve different purposes:

Credential	Length	Who has it	What it's for	Where it's checked
Milestone code	6 digits	The whole class shares codes handed out by the teacher	One-time reward: redeeming it grants 3 random runes, then it's burned	Pins tab (Used column flips to TRUE)
Adventurer PIN	4 digits	Each student sets their own, for their own name	Proves "this is really my account" before rolling, transforming, or crafting	Students tab (PinHash column, salted & hashed)
Teacher passphrase	Any length $\geq$ 3	The teacher only, shared across the whole deployment	Unlocks the Teacher tab: generating codes, managing the item catalog, resetting a PIN	Config tab (TeacherPass row)

The adventurer PIN was built specifically so students can't tamper with each other's runes just by typing a classmate's name. The first time any name is used to roll, transform, or craft, the app asks that person to choose a 4-digit PIN — from then on, that name always asks for it again (once per browser session, since it's cached briefly so students aren't re-typing it constantly). Viewing a name's inventory does not require the PIN; only the three actions that change rune counts do.

Note on "Grant a God Rune": this teacher-only action from earlier revisions no longer exists. God Runes are now earned entirely by students via Transform — see Section 6.1.

4. Glossary, for anyone non-technical

Term	Plain-language meaning
Frontend	The part of the app you actually see and click on — the webpage itself.
Backend	The part running behind the scenes that enforces the rules and stores data. Students never see it directly.
API / endpoint	A specific “request type” the backend understands, like “roll runes” or “add a new item”. The frontend sends one of these every time it needs something done.
Google Apps Script	A free tool from Google that lets you write small programs that live inside a Google Sheet or Doc and can be reached over the internet.
Deployment	A published, stable version of the backend program with its own web address. Editing the code does nothing to the live app until you deploy a new version.
Hash (of a PIN)	A one-way scrambled version of a PIN. The real 4-digit number is never stored or sent back anywhere — only this scrambled fingerprint, which can confirm a guess is correct without ever revealing the original number.
Lock (concurrency lock)	A brief “hold on, someone else is mid-edit” pause the backend uses so two people acting at the same instant can't corrupt each other's data.
Runeword catalog / Runewords tab	The live, editable list of every craftable magic item, stored as its own tab in the Google Sheet. The teacher edits it from the app; nobody edits code to add an item anymore.
Seed / seeding	A one-time “fill in the starting data” step. The Runewords tab is seeded once, the first time it's empty, with a starter catalog — after that, the sheet (not the code) is the source of truth.
localStorage / sessionStorage	Small amounts of data a website can remember inside one specific browser. Used here for convenience settings and the baked-in backend URL override, never for the actual rune or item data.

5. The data model (the Google Sheet)

The Sheet has four tabs. Each row in Students, Pins, or Runewords is one record, exactly like a row in any spreadsheet.

5.1 Students tab — one row per adventurer

Column	Contains
Name	The adventurer's name, exactly as typed. Matching is case-insensitive.
TIR, ITH, NEF, TAL, ORT, THUL, SHAEL, IO, LUM, JAH, BER, ZOD	One column per ordinary rune, holding how many of that rune this student currently owns.
GOD	How many God Runes this student owns — now forged via Transform only (see Section 6.1), never hand-granted.
Crafted	A JSON list of every runeword this student has forged, each with a name and timestamp. e.g. [{"name":"Enigma","at":"2026-09-18T..."}]
PinHash	A salted SHA-256 fingerprint of the student's 4-digit PIN. Never the PIN itself.
PinSalt	A random value mixed into the PIN before hashing, unique per student, so two students with the same PIN don't produce the same hash.
PinFailCount	How many wrong PIN attempts in a row. Resets to 0 on a correct guess.
PinLockUntil	If set, a timestamp before which this student's PIN is temporarily locked out (after 5 wrong attempts in a row).

5.2 Pins tab — one row per milestone code ever generated

Column	Contains
Code	The 6-digit one-time code.
Used	TRUE once someone has redeemed it, otherwise FALSE.
UsedBy	Which adventurer name redeemed it.
CreatedAt / UsedAt	Timestamps for when the teacher generated it and when it was redeemed.

5.3 Config tab — small key/value settings

Currently holds a single row: **TeacherPass** → the shared teacher passphrase, set the first time anyone unlocks the Teacher tab.

5.4 Runewords tab (new) — one row per craftable item — now the single source of truth

This tab replaces the old hardcoded **RUNEWORDS** arrays that used to live, duplicated, in both index.html and Code.gs. It is created and seeded once by the **setup** function (see Section 9.1) from the **RUNEWORD_SEED**

constant in Code.gs, and is never re-seeded after that — from that point on, every add, edit, hide, or delete happens directly on this tab, live, from the Teacher tab's "Manage Items" panel, with no code change or redeploy needed.

Column	Contains
Id	A URL-safe slug (e.g. "enigma"). Auto-generated from the name when an item is added; kept unique by appending -2, -3, etc. if needed.
Name	Display name of the item, e.g. "Enigma".
Slot	Equipment slot / category, e.g. "Chest Armor", "Ring", "Food".
Recipe	A JSON array of rune codes required, e.g. ["BER","NEF","THUL","ORT"]. Can include "GOD" as an ingredient (used by Summarmækir and Ran's Net).
Effect	Free-text description of the item's in-game effect.
Quote	Optional flavor-text quote.
Image	Either blank (falls back to images/{id}.png shipped in the frontend's images/ folder) or a base64 data: URI the teacher uploaded through the item form, client-compressed before upload — see Section 6.2.
Hidden	TRUE/FALSE. Hidden items are excluded from the student-facing Craft grid and Codex, and can't be crafted, but stay in the catalog for the teacher to unhide later.
UpdatedAt	Timestamp of the last add/edit/hide-toggle on this row.

What's still duplicated, and what isn't: the twelve ordinary runes' codes, roll odds, and point values (the RUNES constant — Section 7.1) are still hand-kept in sync between index.html and Code.gs, the same as before, because they're simple and change rarely. Runeword content is the opposite now: it lives only in the Sheet, is fetched live by both the student view (**getRunewords**) and the teacher view (**teacherListRunewords**), and must never be hardcoded back into either file — doing so would make the two disagree with what the Teacher tab shows and edits.

5.5 How concurrency is handled

Every backend action that changes data (rolling, transforming, crafting, generating a code, adding/editing/hiding/deleting an item, etc.) wraps its work in a LockService lock. If two people happen to click at the exact same moment — including a student crafting an item the teacher is simultaneously editing — the second request simply waits its turn (up to 10 seconds) rather than the two writes colliding and corrupting data.

6. How the newer features actually work

6.1 The God Rune formula

God Runes are no longer hand-granted by the teacher. A student forges one themselves, from the Transform tab, by choosing **GOD** as the target rune. Unlike an ordinary Transform (which trades any combination of runes worth enough total points), forging a God Rune uses a **fixed recipe, not a point trade**: the student must destroy **exactly one of each of the twelve ordinary runes** (Tir, Ith, Nef, Tal, Ort, Thul, Shael, Io, Lum, Jah, Ber, Zod) in a single action, which yields exactly one God Rune. Owning extra copies of a rune beyond the one required doesn't help — the backend (**apiTransform** in Code.gs) checks the target is "GOD" first and, if so, requires a quantity of at least 1 for every single code in RUNE_CODES before allowing the trade; any other selection is rejected with "You need one of every ordinary rune to forge a God Rune." Two runewords (Summarmækir and Ran's Net) each require one God Rune in their own recipe.

*This is validated entirely server-side, in the same function that handles ordinary Transforms — there's no separate API action for it. If you ever change this recipe (e.g. to require two of a particular rune instead of one), the change belongs in **apiTransform** in Code.gs, and nowhere in the frontend, since the frontend never decides whether a trade is legal.*

6.2 Manage Items: add / edit / hide / delete runewords

The Teacher tab now has a "Manage Items" panel, listing every runeword (including hidden ones, tagged accordingly) with Edit, Hide/Show, and Delete buttons, plus an "+ Add New Item" button that opens the same form used for editing. The form collects a name, slot, description/effect, an optional flavor quote, an optional photo, and the recipe (clicking rune icons to toggle them on/off, GOD included). All of this maps directly onto the Runewords sheet columns from Section 5.4.

What each button does, end to end:

Action	Frontend call	What the backend does
Add	teacherAddRuneword	Validates the name, slot, and recipe (every code must be a real rune or GOD); slugifies the name into a unique Id; appends a new row to Runewords with Hidden = false.
Edit	teacherEditRuneword	Finds the row by Id; updates only the fields actually sent (so a partial edit can't blank out untouched fields); re-validates the recipe and image size if those changed; stamps UpdatedAt.
Hide / Show	teacherSetRunewordHidden	Flips the Hidden column for that row. Hidden items vanish from the student-facing Craft grid and Codex immediately, but the row — and any data on it — is untouched.
Delete	teacherDeleteRuneword	Permanently removes the row from the sheet after a confirm() prompt in the UI. Students who already crafted that item keep it in their own Crafted list, since that's a frozen name+timestamp snapshot, not a live reference to the Runewords row — but nobody can craft that item again.

After any of these, the frontend re-fetches the catalog (**getRunewords**) and re-renders the Craft tab, Codex, and the Manage Items list itself, so changes are visible everywhere in the same session without a page reload — and on any other device the next time it loads or re-fetches the catalog.

Photos: before an uploaded photo ever reaches the backend, the frontend resizes it (max 480px on the long edge) and JPEG-compresses it on a <canvas> in the browser, backing off quality and then dimensions until the resulting data: URI comfortably clears the backend's 45,000-character cap for a single sheet cell (Code.gs's **MAX_IMAGE_CHARS**). The backend re-checks this limit itself on save (**validateImage_**) in case a caller ever skips the frontend's compression step. If no photo is uploaded, the item falls back to a static file at **images/{id}.png** shipped alongside index.html in the GitHub repo — this is how the 26 seeded runewords get their artwork without bloating the spreadsheet.

7. Game data reference

Important framing change from the previous edition: the runeword table below is the **initial seed** — what RUNEWORD_SEED in Code.gs puts into the Runewords sheet the very first time **setup** runs, if the sheet is empty. It is a historical snapshot, not a live listing. The actual current catalog can differ (items added, edited, hidden, or deleted by the teacher since) and should always be read from the Runewords sheet itself, or the app's Codex tab, not from this document.

7.1 The twelve runes (still hardcoded, still duplicated — see Section 5.4)

Rune	Roll odds	Point value (for Transform)
Tir	14%	2
Ith	14%	2
Nef	12%	5
Tal	12%	5
Ort	10%	9
Thul	10%	9
Shael	8%	14
Io	8%	14
Lum	4%	20
Jah	4%	20
Ber	2%	27
Zod	2%	27

The God Rune is a thirteenth, separate kind: it is never rolled (its odds aren't part of the 100% above) and, as of this revision, is only obtainable by a student forging it via Transform (Section 6.1) — it can no longer be hand-granted by the teacher.

7.2 How a roll is weighted

A roll draws a random number between 0 and 100, then walks down the rune list subtracting each rune's percentage until the number goes negative — whichever rune it lands on is the result. This happens three independent times per Roll action, so duplicates are possible (e.g. two Tir runes in one roll).

7.3 Transform rule (ordinary target)

To forge one ordinary target rune, a student destroys any combination of runes from their own inventory whose point values (Section 7.1) add up to at least the target rune's own point value. Left-over value is not returned — e.g. destroying 28 points of runes to forge a 27-point Ber rune is allowed, but the extra point is simply lost. (Targeting GOD instead follows the fixed-recipe rule in Section 6.1, not this one.)

7.4 Runeword recipes — initial seed (26 items + Sour Key)

Runeword	Slot	Recipe
Enigma	Chest Armor	Ber + Nef + Thul + Ort
Cling	Gloves	Io + Nef + Ith
Prism	Necklace	Tir + Nef + Tal
Silver Tongue of Rizz	Eyewear	Shael + Io
Quicksilver	Harness / Bridle	Lum + Ort + Ith
Charger	Harness / Bridle	Ber + Thul + Tal + Shael
Precious	Ring	Zod + Lum + Thul
Hive	Boots	Ber + Lum + Ort + Ith
Starboard	Oar / Paddle	Io + Nef + Tir
Blade of Carotene	Sword	Io + Nef + Tir
Folly	Scarf	Zod + Shael + Ort
Insolence	Weapon	Lum + Shael + Ort + Tal
Sage	Earrings	Tir + Nef + Ort + Shael
Lark	Scarf	Ith + Tal + Thul + Io
Inferno	Dish	Zod + Io + Thul
Socks of Blocks	Socks	Zod + Jah + Io + Tal
Invigorated by the North	Pendant / Necklace	Lum + Shael + Tal
Bramble	Leg Armor	Shael + Ort + Nef + Tir
Crescent Moon	Musical Instrument	Ber + Shael + Tal + Ith
Harmony	Head Armor	Lum + Io + Tal + Tir
Summarmækir	Weapon	1 God Rune + Zod + Jah + Io
Ran's Net	Weapon	1 God Rune + Ber + Lum + Shael
Herofeller	Scabbard	Tir + Tal + Ort + Io
Coolth	Pillow	Tir + Thul + Shael + Lum
Amulet of Misfortune	Necklace	Tir + Shael + Thul + Nef
Stuffed Buddy	Doll	Ith + Nef + Thul + Io
Sour Key	Food	Shael + Io + Jah + Ber, ×1 (Small) / ×2 (Medium) / ×3 (Large)

*Sour Key is handled by its own dedicated action (**craftSour**) rather than living as a normal Runewords row, since it scales by a size multiplier rather than being a single fixed recipe — this is unchanged from previous revisions and is still hardcoded in Code.gs's SOUR_KEY_BASE constant, not editable from Manage Items.*

8. Full API reference

The frontend talks to the backend with a single web address (the Apps Script deployment URL, baked into `index.html` as a default and overridable via the gear-icon settings). Every request is an HTTP POST with a JSON body containing an **action** field and whatever parameters that action needs. Every response is JSON shaped as either `{"ok":true,"data":...}` or `{"ok":false,"error":"..."}`.

action	Parameters	What it does	PIN required?
checkName	name	Reports whether this name already has a PIN set (used to decide whether to show the “set a PIN” or “enter your PIN” prompt).	No
claimPin	name, authPin	Sets a brand-new 4-digit PIN for a name that doesn't have one yet.	No
verifyPin	name, authPin	Checks a PIN is correct, without doing anything else.	—
getStudent	name	Returns a student's current inventory and crafted-items list. View only.	No
roll	name, code, authPin	Redeems a 6-digit milestone code for 3 random runes.	Yes
transform	name, target, selection, authPin	Destroys the chosen runes (selection) to forge one target rune — or, if target is “GOD”, forges a God Rune from the fixed 1-of-each recipe (Section 6.1).	Yes
craft	name, itemId, authPin	Destroys a runeword's recipe runes and adds the item to Crafted.	Yes
craftSour	name, mult, authPin	Same as craft, but for the size-scaling Sour Key (mult is 1, 2, or 3).	Yes
getRunewords	—	New. Returns every visible (non-hidden) runeword. Called by the frontend on load to populate Craft and Codex.	No
teacherSetup	pass	Sets the shared teacher passphrase (only works once, when none is set).	—
teacherLogin	pass	Checks the teacher passphrase is correct.	—
teacherGenerate Pin	pass	Creates a new, unused 6-digit milestone code.	—
teacherListRunewords	pass	New. Returns every runeword, including hidden ones, for the Manage Items panel.	—
teacherAddRuneword	pass, name, slot, recipe, effect, quote, image	New. Validates and appends a new runeword row.	—
teacherEditRuneword	pass, id, (any of: name, slot, recipe, effect, quote, image)	New. Updates only the fields provided on an existing runeword row.	—

action	Parameters	What it does	PIN required?
teacherSetRunewordHidden	pass, id, hidden	New. Shows or hides a runeword from students without deleting it.	—
teacherDeleteRuneword	pass, id	New. Permanently removes a runeword row. Irreversible.	—
teacherListStudents	pass	Returns every student's totals, crafted count, and whether a PIN is set.	—
teacherResetPin	pass, name	Clears a student's PIN so they can set a new one.	—
teacherReset	pass	Wipes all student and milestone-code data. Irreversible. (Does not touch Runewords.)	—

Removed in this revision: **teacherGrantGod** (pass, name) no longer exists. God Runes are earned only through the Transform action's fixed recipe (Section 6.1).

All “teacher...” actions require the correct pass value in place of a student PIN; there is no separate teacher session token — the passphrase is simply re-sent with every teacher request.

9. Notes for future Claude sessions

If you (a future instance of Claude) are being asked to modify this app, here is what you need to know before touching anything.

9.1 Files and where they live

File	Role	How to change it live
index.html	The real, multi-device frontend. Single self-contained file: HTML, CSS, and JavaScript all inline, no build step. Includes a baked-in <code>DEFAULT_API_URL</code> constant so first-time visitors connect automatically without pasting a URL.	Edit the file, commit, and git push to the GitHub repo. Render auto-redeploys on push — no manual re-upload, unlike the old illswap.net static-hosting setup.
images/	PNG artwork for the seeded runewords, referenced as <code>images/{id}.png</code> when a Runewords row has no uploaded photo.	Add/replace files in this folder in the repo and push, same as <code>index.html</code> .
Code.gs	The backend. Lives inside a Google Apps Script project bound to the teacher's Google Sheet.	Edit the code in the Apps Script editor, then Deploy → Manage deployments → pencil icon → Version: New version → Deploy. Editing alone does NOT affect the live app, and this is the single most common way a fix silently “doesn't work.” Editing an existing deployment this way keeps the same <code>/exec</code> URL; creating a brand new deployment instead would generate a different URL and require updating <code>DEFAULT_API_URL</code> in <code>index.html</code> to match — always prefer editing the existing one.
runic-system.html	A legacy, fully offline fallback version that stores everything in browser <code>localStorage</code> instead of the Sheet. No multi-device sync, no Runewords tab, no Manage Items. Kept only as a no-setup emergency option.	Standalone; unrelated to everything above. Only touch if asked.

9.2 Rules to respect when changing game content

- **Runewords no longer need edits in two files.** Add, edit, hide, or delete a runeword through the app's Manage Items panel (or by editing the Runewords sheet directly) — never hardcode a runeword back into `index.html` or `Code.gs`. If a future request asks you to “add a new item” in code, that's very likely the wrong approach now; point out that Manage Items exists instead, unless there's a specific reason to bypass it.
- **The twelve base runes are the one exception that's still duplicated.** If you add, remove, or reweight a rune itself (not a runeword), you must edit the `RUNES` array in both `index.html` and `Code.gs`, exactly as before this revision — they must stay identical in codes, roll odds, and point values.
- After any `Code.gs` edit, remind whoever is deploying that a new deployment version is required (Section 9.1).

- Never move game-rule validation (roll odds, transform math, the God Rune recipe, craft requirements) into the frontend only. The backend must remain the source of truth, since the frontend is fully visible and editable by anyone with browser developer tools.
- The Sheet's column headers are load-bearing: `readStudentPublic_`/`readStudentAuth_`, `writeInventoryDelta_`/`writeFields_`, and the new `rowToRuneword_`/`writeRunewordFields_` helpers all look columns up by header name, not position. Renaming a header breaks the mapping. Adding a new header is safe (`ensureColumns_` appends without disturbing existing ones).
- If you add a new field to a runeword (say, a rarity tag), it needs: a new column in `RUNEWORD_HEADERS`, handling in `rowToRuneword_` and `writeRunewordFields_`, a form field in the frontend's item form, and display logic wherever runewords are rendered (Craft, Codex, Manage Items list) — the data itself will only ever live in the sheet.

9.3 What NOT to do

- Do not publish `index.html` through Claude's Artifact tool as the “live” version — Claude's hosted artifact preview blocks outbound network calls to external domains like `script.google.com`, so the backend connection will silently fail there. This file must be opened as a real file (pushed to GitHub → served by Render, or opened locally) to function.
- Do not store the real 4-digit PIN or the teacher passphrase anywhere except as intended (`PinHash`/`PinSalt` for PINs, plain in `Config` for the teacher passphrase, which is an accepted, documented trade-off for a classroom-grade tool, not a security oversight).
- Do not remove the `LockService` locks around mutating `Code.gs` functions (including the newer `teacherAddRuneword` / `teacherEditRuneword` / `teacherSetRunewordHidden` / `teacherDeleteRuneword`); they are the only thing preventing two simultaneous requests from corrupting data.
- Do not raise `MAX_IMAGE_CHARS` casually to “fix” an image upload failure — it exists because a single Google Sheets cell has a hard ~50,000-character ceiling; if photos genuinely need to be larger or higher-quality, that requires a different storage approach (e.g. Google Drive with a stored file URL instead of an inline data: URI), not just a bigger number.

9.4 Good first questions to ask the teacher before changing something

- Is this a runeword content change (new/edited item)? That's now a Manage Items task in the app itself — it may not need you at all.
- Is this a base-rune change (new rune, new odds) or a behavior change (new feature, new screen, new validation rule)? Base-rune changes need edits in two files; behavior changes are usually one file.
- Does this change need to preserve existing student data or the current runeword catalog? (Almost always yes — avoid anything that would require wiping the Students, Pins, or Runewords tabs.)
- Will the teacher need to redeploy the Apps Script, and do they still remember how (Section 9.1)?
- If a `Code.gs` change is needed, will it require a new deployment (keep the same URL) or has a new deployment already been created with a different URL that needs to be re-baked into `index.html`'s `DEFAULT_API_URL`? This exact mix-up has already caused a live outage once — always confirm which deployment the live app is actually pointed at before assuming a redeploy fixed something.

9.5 Known limitations, by design

Limitation	Why it's acceptable here
Google Sheets isn't built for heavy concurrent traffic.	Fine for a class of dozens acting occasionally; would need a real database if this ever scaled to hundreds of simultaneous users.
The teacher passphrase and student names travel as plain JSON over HTTPS.	Encrypted in transit by HTTPS; acceptable for a fun in-class tool with no sensitive personal data, not designed for anything higher-stakes.
Any two people can independently create an account under the same name	if the second person gets there before the first sets a PIN. First to set a PIN "owns" that name from then on — an accepted trade-off for keeping the system simple and account-free.
No true user accounts (email/password, etc.).	Deliberately avoided so any student can start playing immediately with just a name, no sign-up flow.
A single deleted or renamed image file in images/ silently breaks a seeded item's photo.	There's no validation that images/{id}.png exists; the tag just fails to render. Acceptable since it's cosmetic — crafting itself doesn't depend on the image loading.

10. One-paragraph mental model

If you remember only one thing from this document:

The webpage is just a display. Every rule — who can act, what a roll produces, what a craft costs, what a God Rune requires — lives in Code.gs, which is the only thing allowed to read or write the Google Sheet. The Sheet is the entire database, and that now includes not just student and code data but the runeword catalog itself. Nothing is real until Code.gs says it's real — and nothing about what items exist is real until the Runewords tab says so.

Document generated for the teacher's ongoing use with Claude. Keep this alongside index.html, the images/ folder, and Code.gs, and share it at the start of any future conversation asking Claude to modify the Runic System.